

UML-діаграми проєкту

Remote File Folder Client

(спрощений аналог Google Drive)

Уточкіна Яна

Етап: Проєктування архітектури та поведінки системи
12 вересня 2026 р.

Анотація

У документі представлено комплект UML-діаграм для проєкту **Remote File Folder Client** — застосунку для роботи з віддаленою файловою текою (спрощений аналог Google Drive) з веб- та десктоп-клієнтами, Go-бекендом, PostgreSQL та синхронізацією файлів за принципом diff-маніфестів. Діаграми впорядковано відповідно до життєвого циклу проєктування: від аналізу вимог і високорівневої архітектури до деталізованого проєктування окремих підсистем. Усі діаграми побудовано в нотатції Mermaid.

Зміст

1 Аналіз вимог	2
2 Архітектурне проєктування (високий рівень)	2
2.1 Діаграма прецедентів	2
2.2 Діаграма класів (загальна структура)	4
2.3 Діаграма розгортання (загальна архітектура)	7
3 Деталізоване проєктування	8
3.1 VOPC-діаграма класів для прецедента “Синхронізувати локальну теку з віддаленою”	9
3.2 Діаграма об’єктів (знімок конфлікту)	11
3.3 Діаграми взаємодії (Sequence & Communication Diagrams) . . .	13
3.4 Діаграма станів	21
3.5 Діаграма компонентів	23

3.6	Діаграма пакетів	24
3.7	Діаграма активності	24
4	Висновки	25

1. Аналіз вимог

Система надає користувачу автентифікований доступ до персональної віддаленої теки (“віртуального диска”), у якій він може переглядати список файлів разом з їх атрибутами (ім’я, дата створення, дата модифікації, автор вивантаження, автор останньої редакції), вивантажувати, завантажувати та видаляти файли, а також синхронізувати обраний локальний каталог із віддаленим.

Ключові функціональні вимоги:

- автентифікація користувача на віддаленому сервері;
- перегляд атрибутів файлів із можливістю показу/приховування стовпців (стовпець імені файлу приховати не можна);
- перегляд вмісту файлів окремих типів: `.java` – як текст, `.png` – як зображення;
- сортування файлів за іменем користувача, який востаннє редагував файл (за зростанням / спаданням);
- фільтрація файлів: усі файли / лише `.cs` / лише `.jpg`;
- синхронізація локальної теки з віддаленою на основі порівняння маніфестів (`name`, `size`, `mtime`, `sha256`) та обробка конфліктів редагування.

Архітектурно система складається з React-клієнта (веб), Swift-клієнта (macOS, з повноцінною фоновією синхронізацією через FSEvents), Go API-сервера (шари `handler` / `service` / `repository`), PostgreSQL для метаданих та абстрактного сховища файлів (локальний диск або S3-сумісне сховище), розгорнутих на домашньому Ubuntu-сервері та опублікованих через Cloudflare Tunnel.

На основі цих вимог у наступних розділах побудовано діаграму прецедентів, а також архітектурні та деталізовані діаграми, що описують структуру і поведінку системи.

2. Архітектурне проєктування (високий рівень)

На цьому етапі визначається загальна архітектура системи: хто є її користувачами і які сценарії вона підтримує (діаграма прецедентів), яка загальна структура даних (діаграма класів високого рівня) та яка фізична топологія розгортання (діаграма розгортання).

2.1 Діаграма прецедентів

Діаграма (рис. 1) показує єдиного актора *Користувача*, який працює із системою через веб- або десктоп-клієнт, та основні прецеденти, що відповідають функціональним вимогам розділу 1. Частина операцій над файлами включають («include») прецедент “Автентифікуватися”, оскільки до-

ступ до робочої області можливий лише після успішного входу. Перегляд вмісту файлу *розширюється* («extend») двома альтернативними сценаріями відображення (. java / . png), а синхронізація – сценарієм розв’язання конфлікту, що активується лише за наявності розбіжностей.

Примітка щодо нотації: Mermaid не має вбудованого типу діаграми прецедентів, тому актора, прецеденти та межу системи побудовано засобами flowchart (актор – прямокутник зі заокругленими кутами, прецеденти – “stadium”-форма, межа системи – subgraph), із явним підписом стереотипів «include»/«extend» на зв’язках.

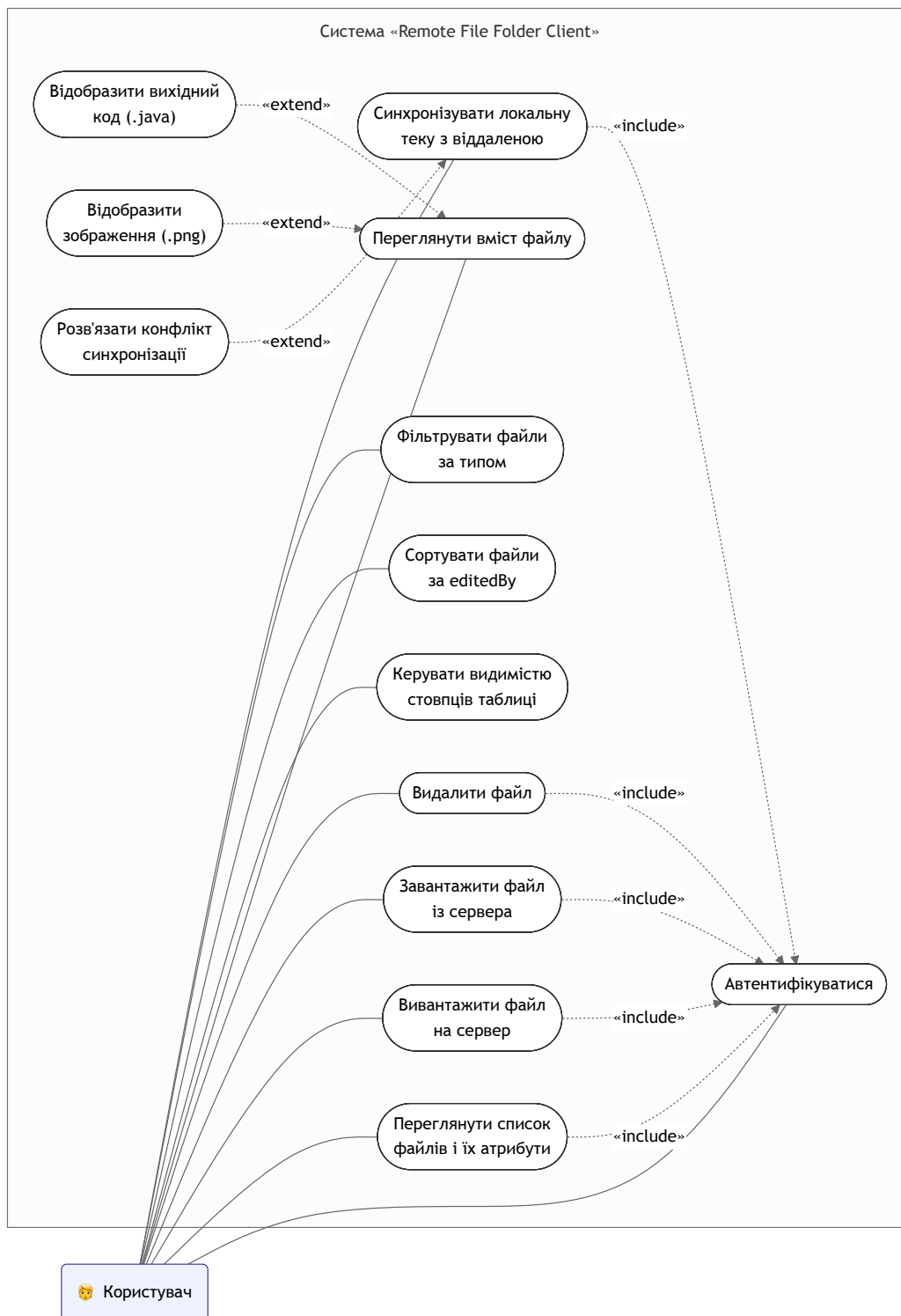


Рис. 1: Діаграма прецедентів системи Remote File Folder Client

2.2 Діаграма класів (загальна структура)

Рис. 2 відображає загальну структуру даних та основні взаємозв'язки між доменними класами системи, без деталізації методів окремих сценаріїв: сутності User та File, сервісний шар (AuthService, FileService, SyncDiffService),

шар доступу до даних (FileRepository) та абстракцію сховища файлів FileStorage, реалізовану локальним сховищем на диску та S3-сумісним сховищем – відповідно до принципу “сховище за інтерфейсом”, закладеного в вимогах (розділ 5.2, 5.5).

7

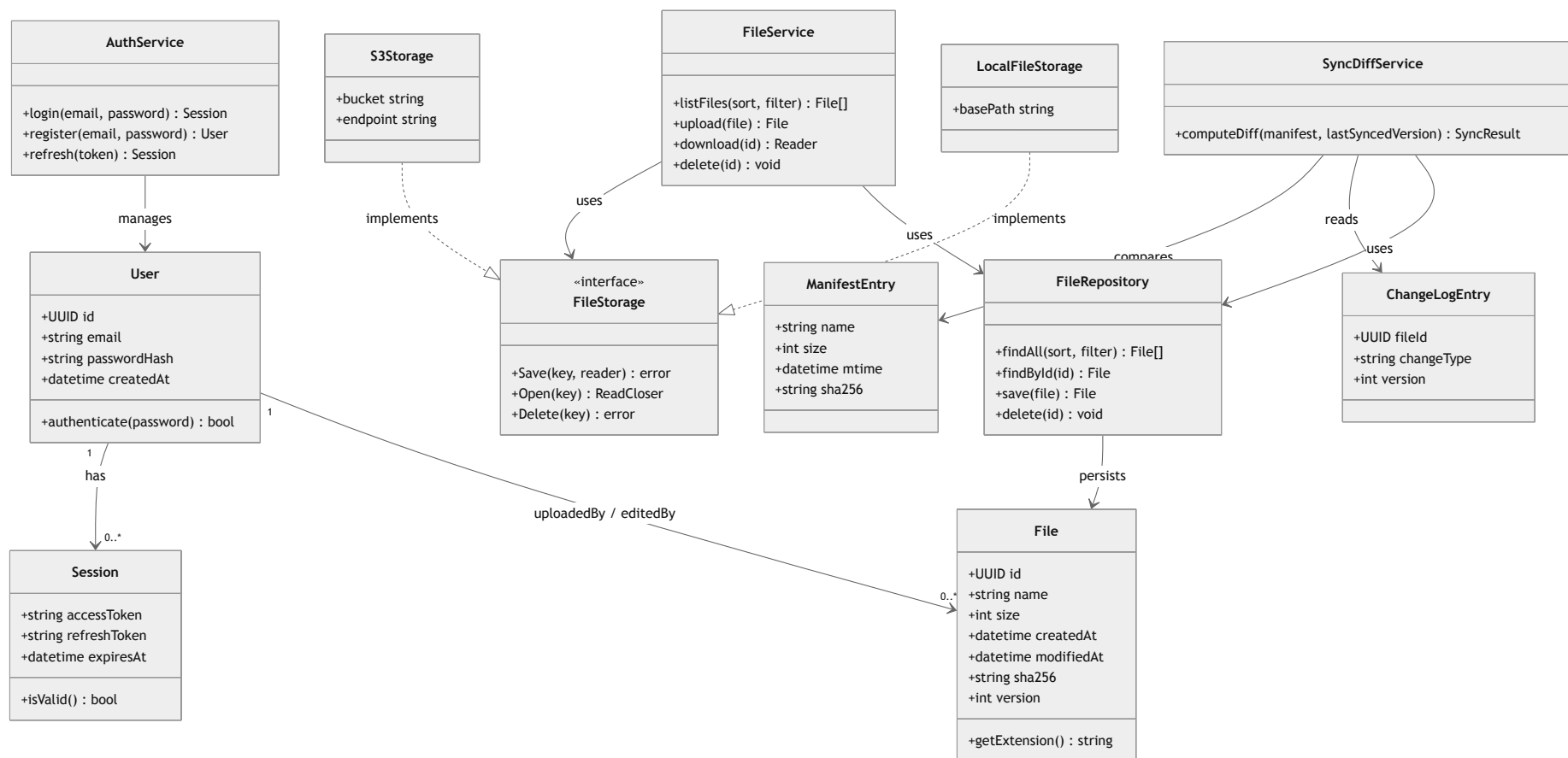


Рис. 2: Діаграма класів високого рівня (доменна модель)

2.3 Діаграма розгортання (загальна архітектура)

Рис. 3 відображає фізичну топологію системи: клієнтські пристрої (браузер, macOS), периферію Cloudflare (термінація TLS, публічний DNS) та домашній Ubuntu-сервер, що через Docker Compose підіймає три контейнери – `cloudflared` (вихідний тунель без відкриття портів), `api` (Go-бінарник) та `db` (PostgreSQL) – разом із постійним томом `pgdata` та `bind-mount` каталогом для файлового сховища.

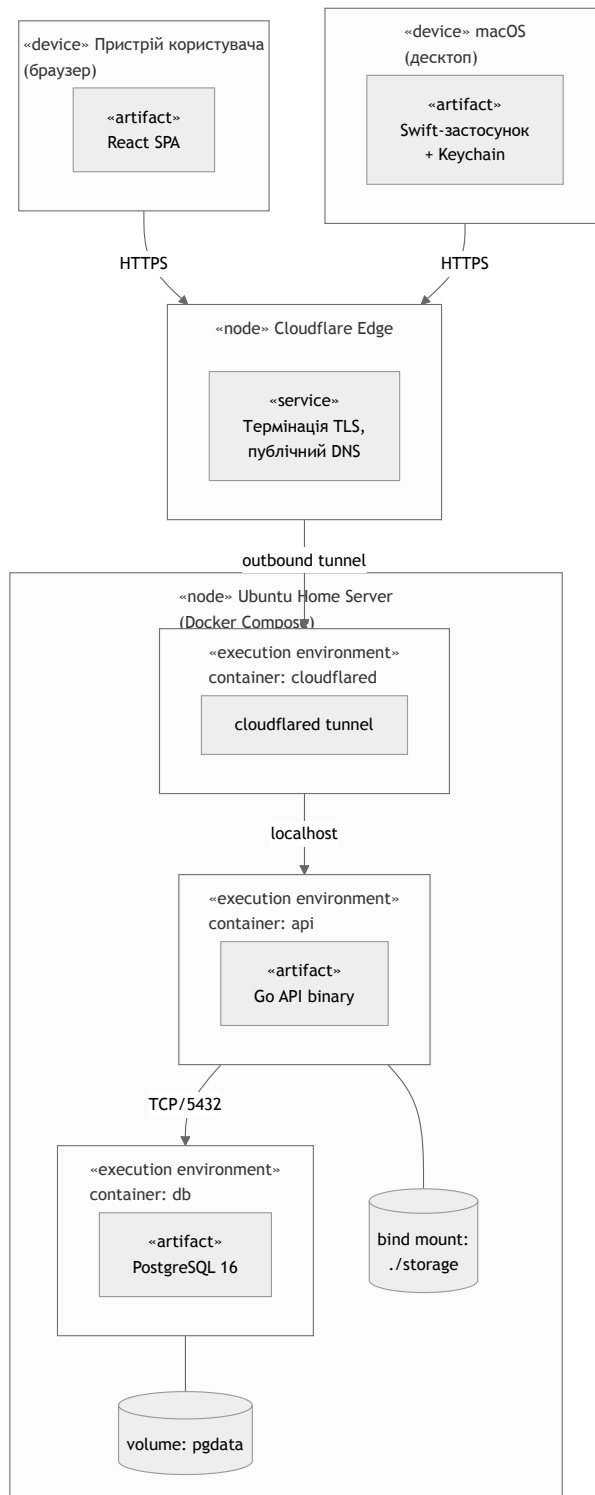


Рис. 3: Діаграма розгортання

3. Деталізоване проєктування

Деталізоване проєктування розкриває внутрішню структуру та поведінку окремих підсистем: класи, що беруть участь у реалізації конкретного прецедента (VOPC-діаграма), конкретний знімок станів об'єктів під час кон-

флікту (діаграма об'єктів), взаємодію об'єктів у часі – як у часовій (sequence), так і в структурній (communication) нотації – поведінку файлу залежно від стану синхронізації (діаграма станів), структуру програмних компонентів (діаграма компонентів), організацію коду за модулями (діаграма пакетів) та логіку бізнес-процесу синхронізації (діаграма активності).

3.1 VOPC-діаграма класів для прецедента “Синхронізувати локальну теку з віддаленою”

Згідно з варіантом завдання, як прецедент для VOPC-діаграми (*View Of Participating Classes*) обрано “**Синхронізувати локальну теку з віддаленою**” – найскладніший і найбільш детально описаний у вимогах сценарій (розділ 6 вимог). Класи розподілено за стандартними стереотипами аналізу:

- «boundary» – елементи взаємодії з користувачем: спостерігач за текою (FolderWatcherUI), кнопка ручної синхронізації (SyncNowButtonUI), діалог розв'язання конфлікту (ConflictDialogUI);
- «control» – керуюча логіка: SyncController (клієнт), SyncDiffService (сервер, обчислення diff), ConflictResolver (застосування вибору користувача);
- «entity» – дані предметної області: ManifestEntry, SyncCursor, File, ChangeLogEntry.

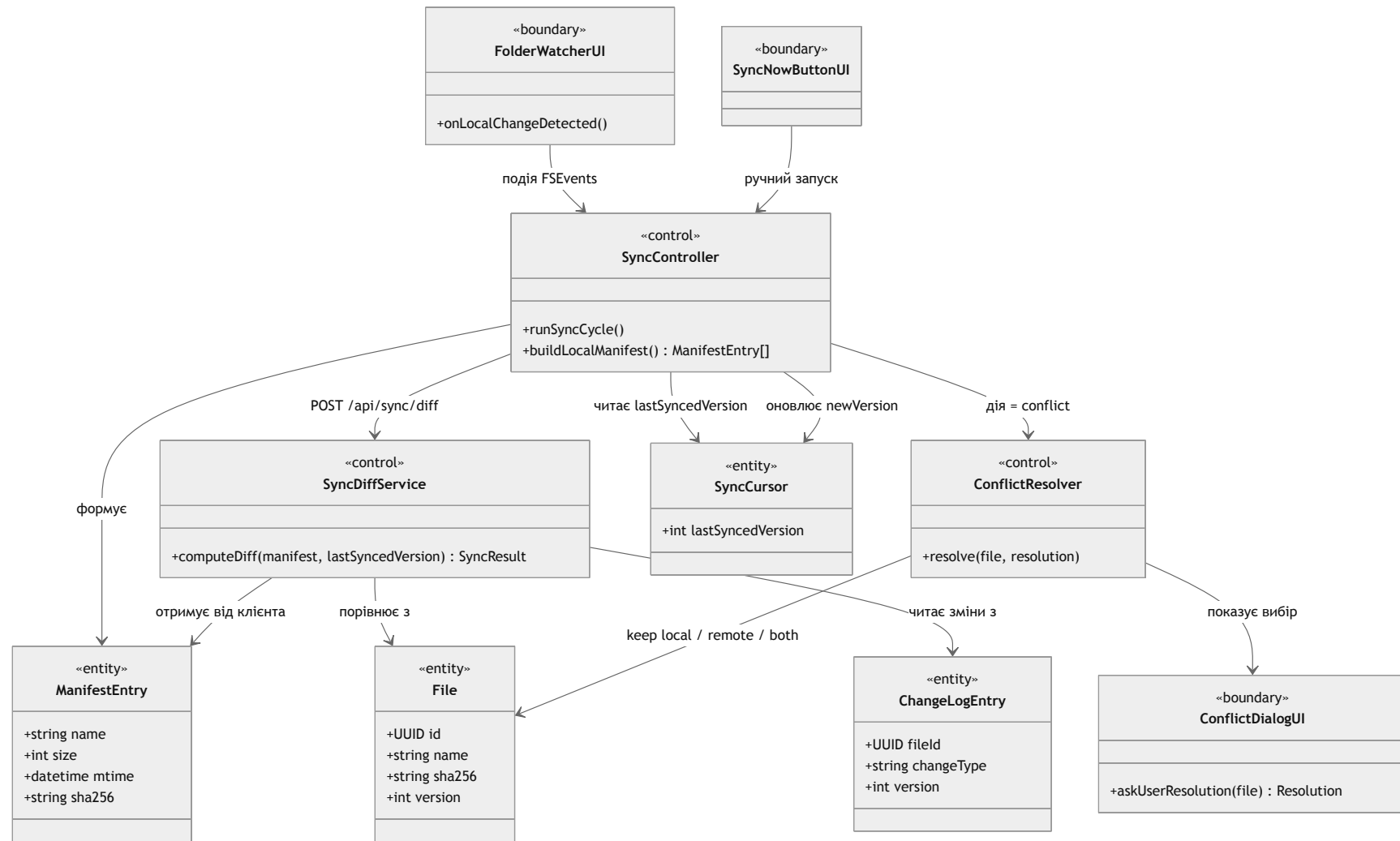


Рис. 4: VOPC-діаграма класів для прецедента “Синхронізувати локальну теку з віддаленою”

3.2 Діаграма об'єктів (знімок конфлікту)

На відміну від класових діаграм, що описують структуру типів, діаграма об'єктів (рис. 5) фіксує *конкретний знімок* екземплярів під час виконання – одну мить сценарію синхронізації, коли виявлено конфлікт (розділ 6.2 вимог, випадок “exists, hash differs”). Локальний екземпляр `localFile` (версія 44, обчислена на клієнті) та серверний `remoteFile` (версія 45) мають однакове ім'я й розмір-ідентичність за `id`, але різняться за `sha256` – це і є ознака редагування з обох боків. Запис `cll : ChangeLogEntry` підтверджує, що версія 45 новіша за `lastSyncedVersion = 42`, збережену в курсорі клієнта, тобто зміна на сервері відбулася вже після останньої успішної синхронізації.

Примітка щодо нотації: Mermaid не має вбудованого типу діаграми об'єктів, тому екземпляри побудовано засобами `flowchart`: заголовок кожного об'єкта підписано за конвенцією UML як ім'я_екземпляра : Клас, а нижче через розділювач перелічено конкретні значення атрибутів.

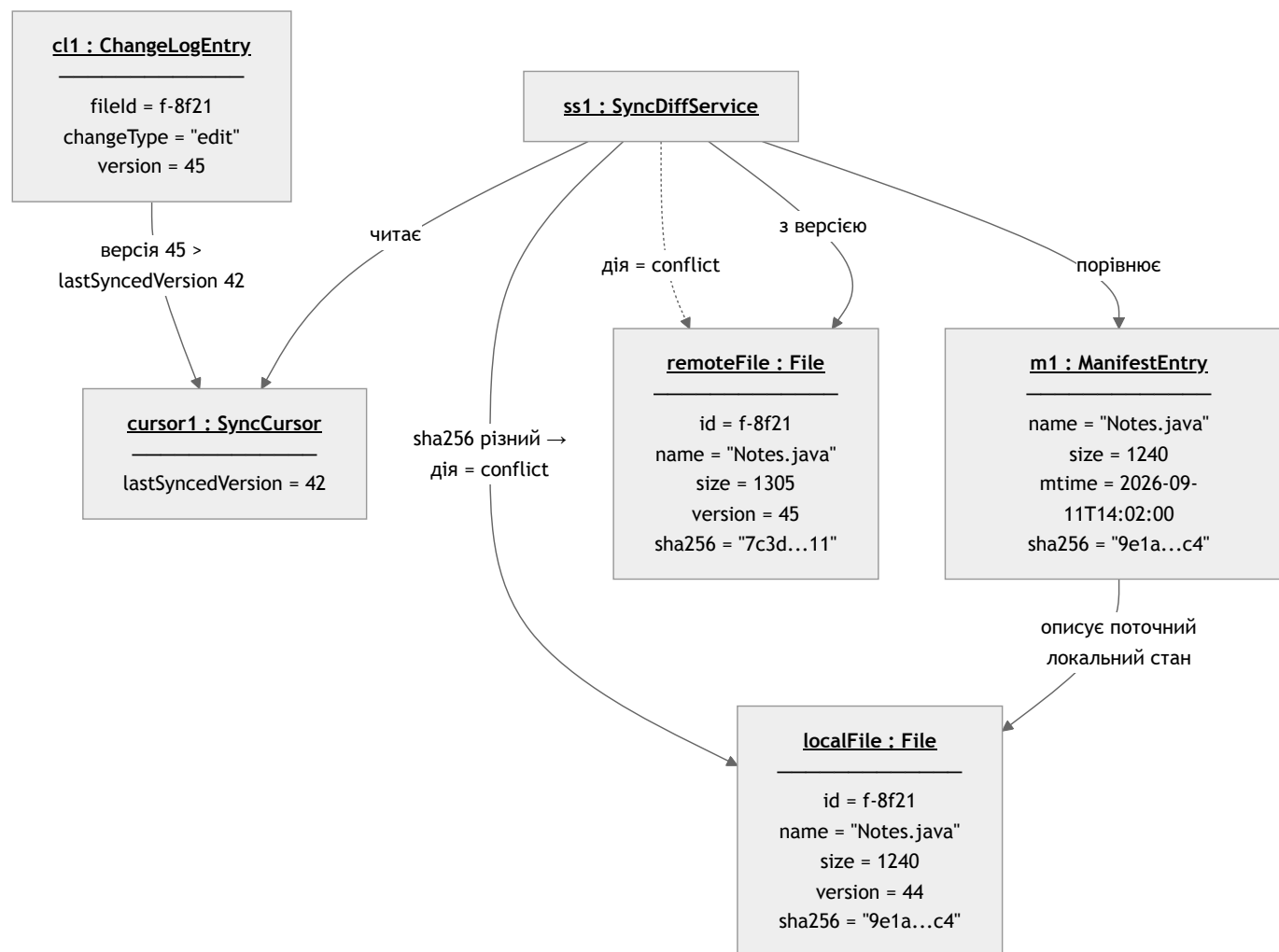


Рис. 5: Діаграма об'єктів: знімок стану під час конфлікту редагування

3.3 Діаграми взаємодії (Sequence & Communication Diagrams)

Наведено три діаграми послідовності, що ілюструють ключові сценарії взаємодії клієнта, Go API-сервера та бази даних, а також комунікаційну діаграму як альтернативне представлення одного з цих сценаріїв.

Автентифікація

Рис. 6 показує послідовність перевірки облікових даних користувача та видачі пари токенів (access/refresh) відповідно до опису автентифікації у вимогах (розділ 1, 5.2).

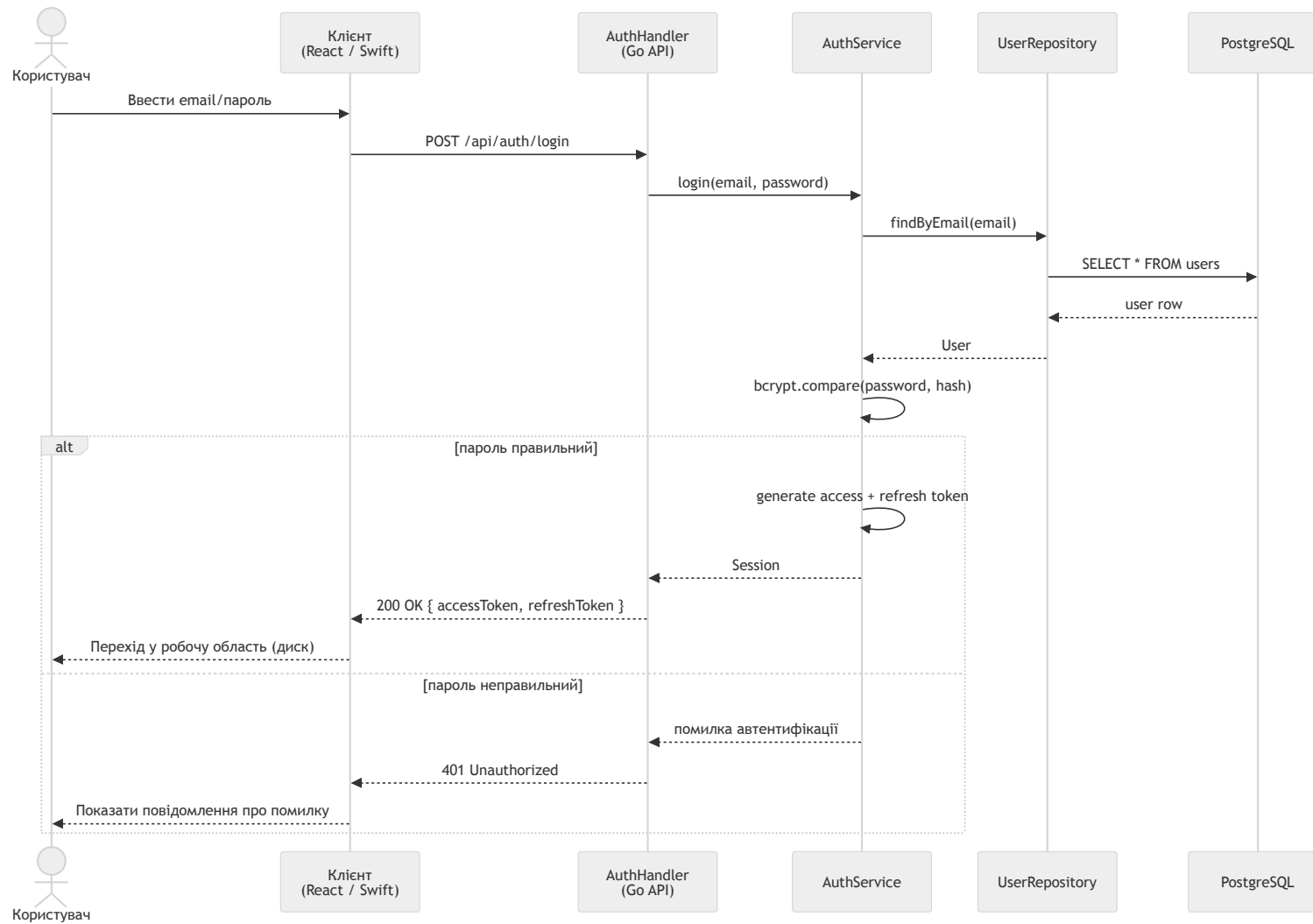


Рис. 6: Діаграма послідовності: автентифікація користувача

Вивантаження файлу

Рис. 7 ілюструє шлях запиту від клієнта через шар обробників (handler) до сервісного шару, який послідовно записує вміст файлу у сховище (FileStorage), а після підтвердження запису – метадані файлу у PostgreSQL через FileRepository.

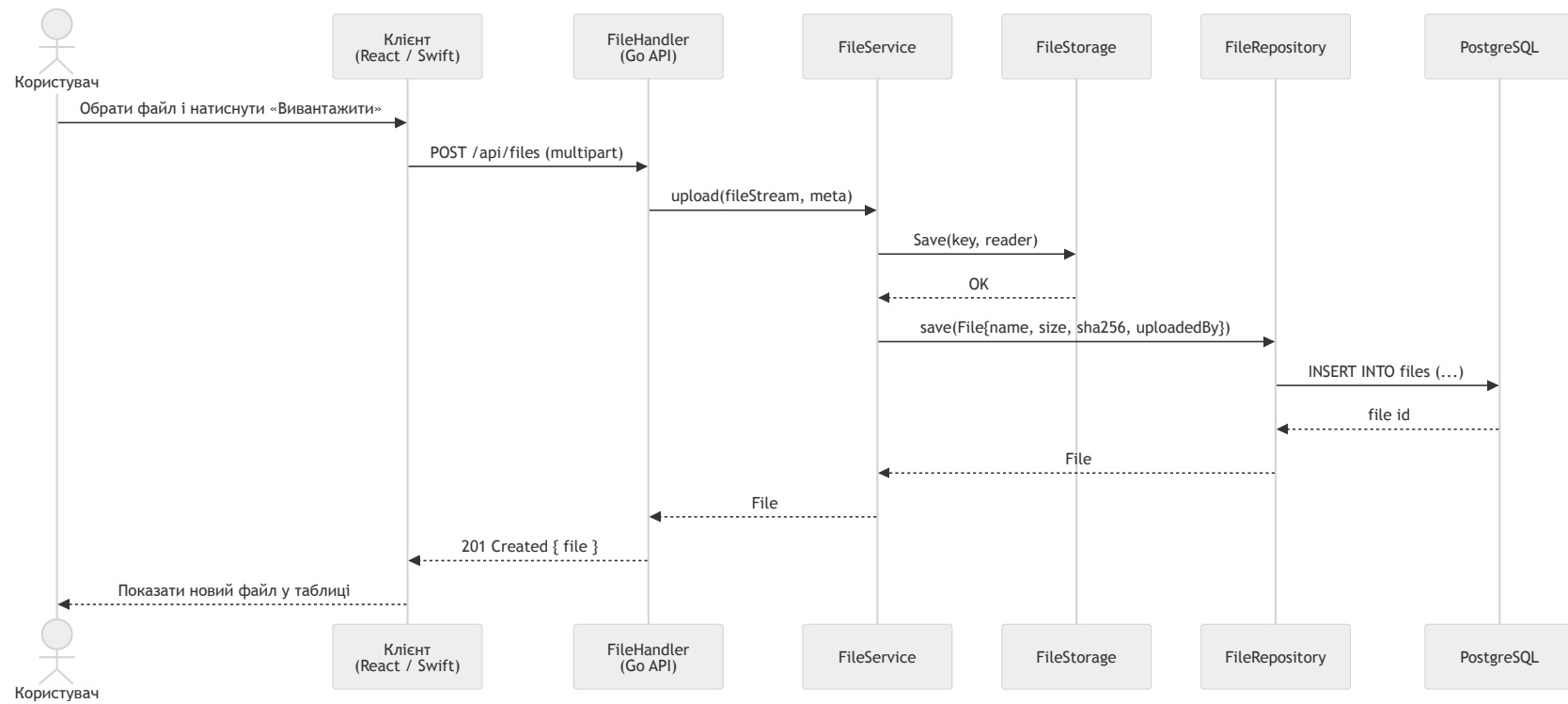


Рис. 7: Діаграма послідовності: вивантаження файлу на сервер

Синхронізація та розв'язання конфлікту

Рис. 8 деталізує протокол `POST /api/sync/diff` (розділ 6.6 вимог): клієнт надсилає локальний маніфест і `lastSyncedVersion`, сервер повертає перелік дій для кожного файлу, а конфліктні файли обробляються через запит вибору користувача (`keep local / keep remote / keep both`).

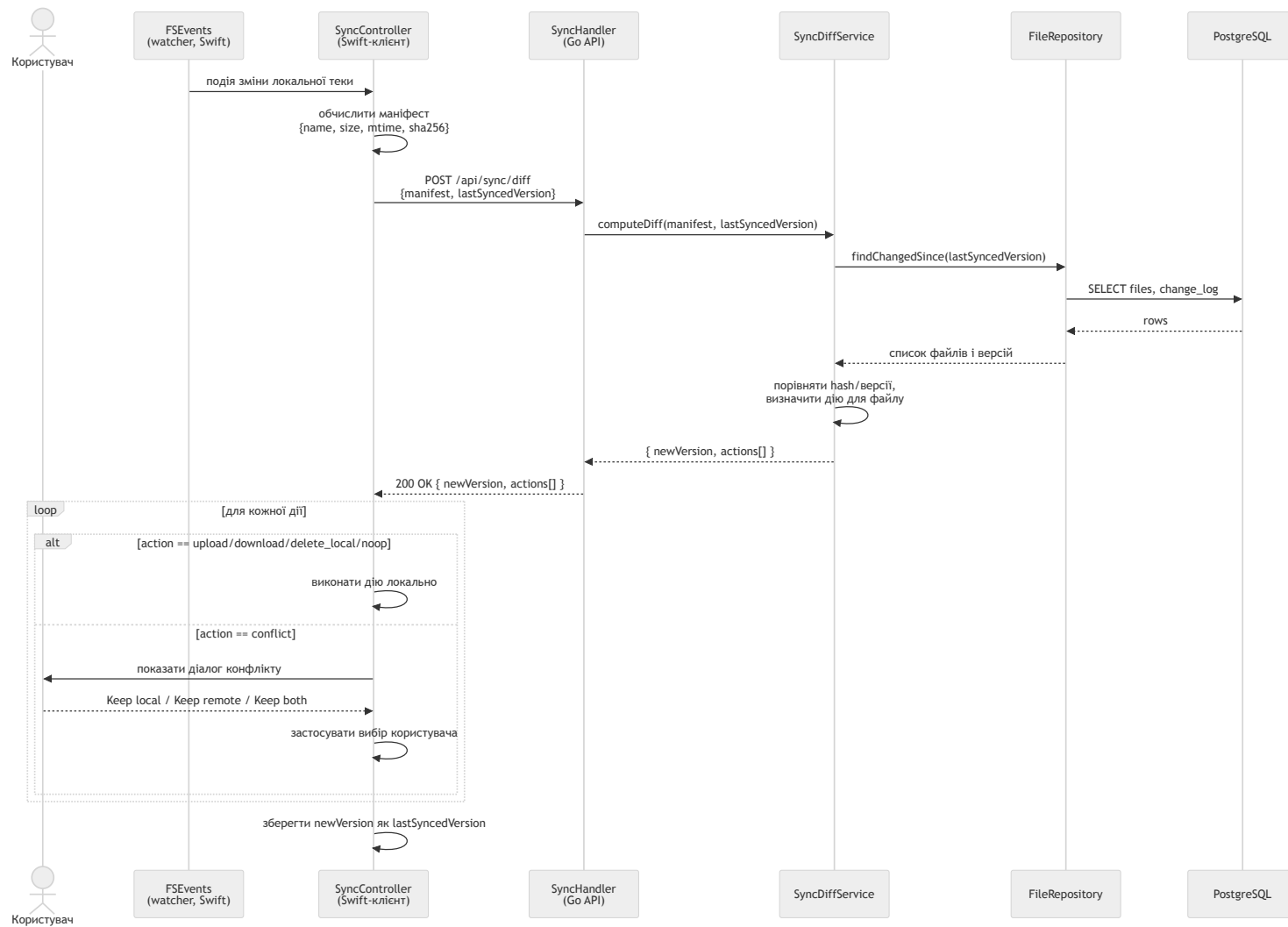


Рис. 8: Діаграма послідовності: синхронізація та розв'язання конфлікту

Той самий сценарій у нотації комунікаційної діаграми

Рис. 9 зображує **той самий** сценарій синхронізації та розв'язання конфлікту, що й рис. 8, але в нотації комунікаційної (communication) діаграми: акцент зміщено з часової осі на статичні зв'язки між об'єктами, а порядок повідомлень передається нумерацією (з десятковою вкладеністю для повідомлень усередині умовних гілок – наприклад, 5.1 - -5.3 для взаємодії з користувачем у разі конфлікту). Обидві діаграми узгоджені між собою – використовують ті самі об'єкти й повідомлення – і демонструють два стандартні способи показати одну взаємодію.

Примітка щодо нотації: Mermaid не має вбудованого типу комунікаційної діаграми, тому вона побудована засобами flowchart: об'єкти – вузли графа, а нумеровані підписи на зв'язках відповідають повідомленням.

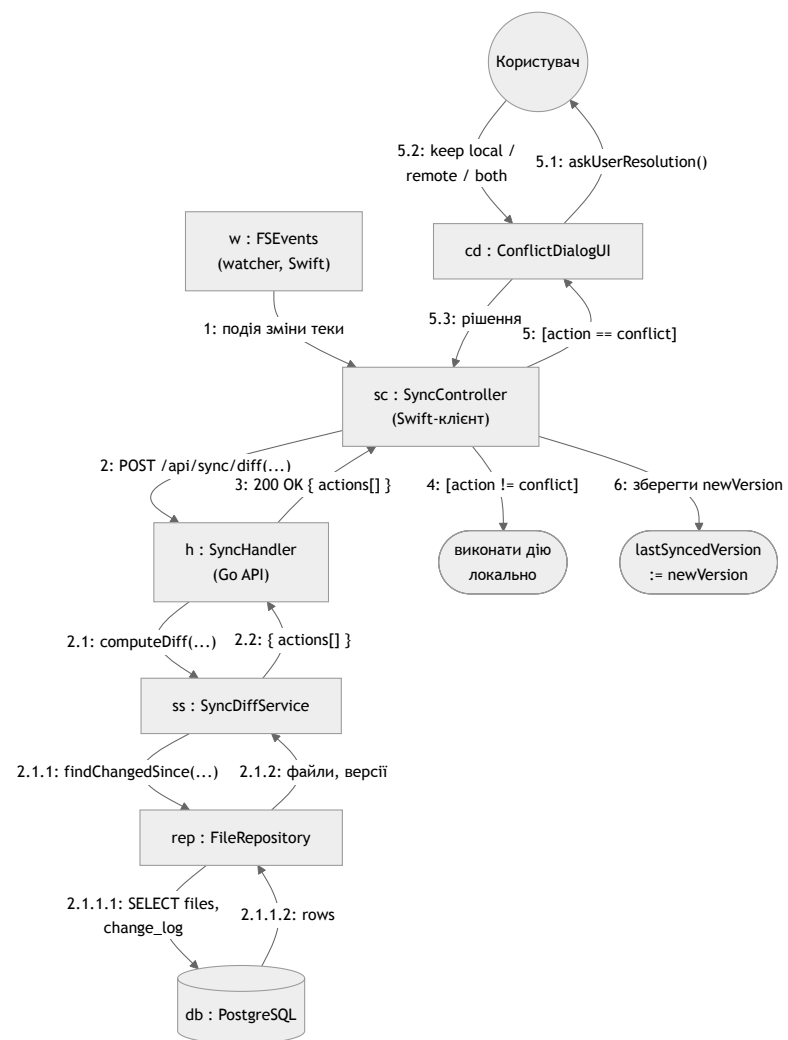


Рис. 9: Комунікаційна діаграма: синхронізація та розв'язання конфлікту

3.4 Діаграма станів

Рис. 10 описує поведінку сутності *File* залежно від стану синхронізації: від локального/віддаленого створення, через передачу (Uploading/Downloading), до синхронізованого стану, можливих локальних чи віддалених змін і, за одночасної зміни з обох боків, – конфлікту, що розв’язується користувачем. Діаграма безпосередньо реалізує чотири випадки порівняння маніфестів із розділу 6.2 вимог (upload / download / no-op / conflict) разом з обробкою видалень через версійний курсор (розділ 6.3).

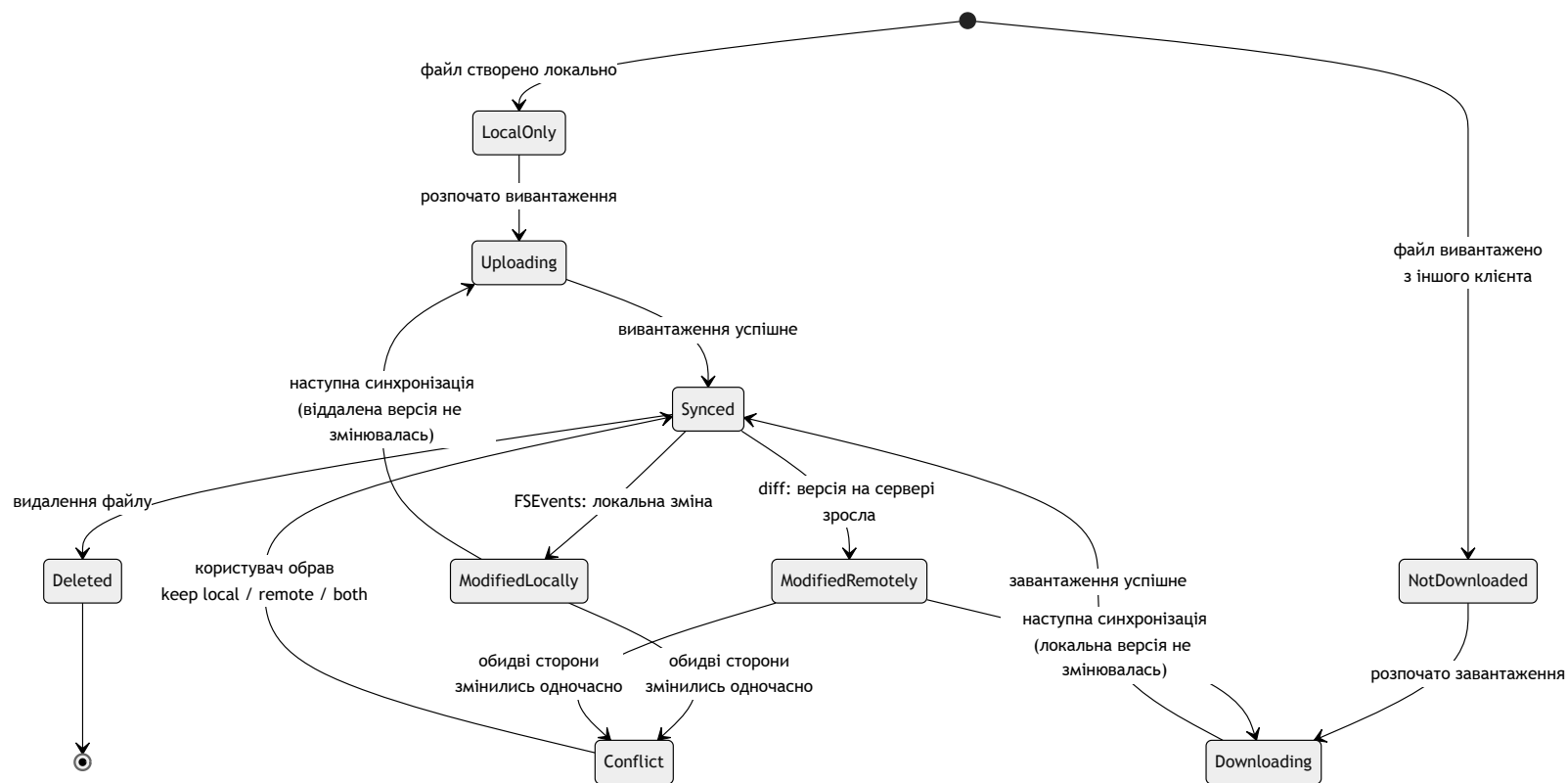


Рис. 10: Діаграма станів файлу в процесі синхронізації

3.5 Діаграма компонентів

Рис. 11 деталізує програмні компоненти системи та їх залежності: клієнтські застосунки, тунель cloudflared, а всередині Go API-сервера – шари Handler → Service → Repository, а також незалежний від конкретної реалізації інтерфейс FileStorage, що дозволяє замінити локальне сховище на S3-сумісне без зміни бізнес-логіки (розділ 5.2, 5.5 вимог).

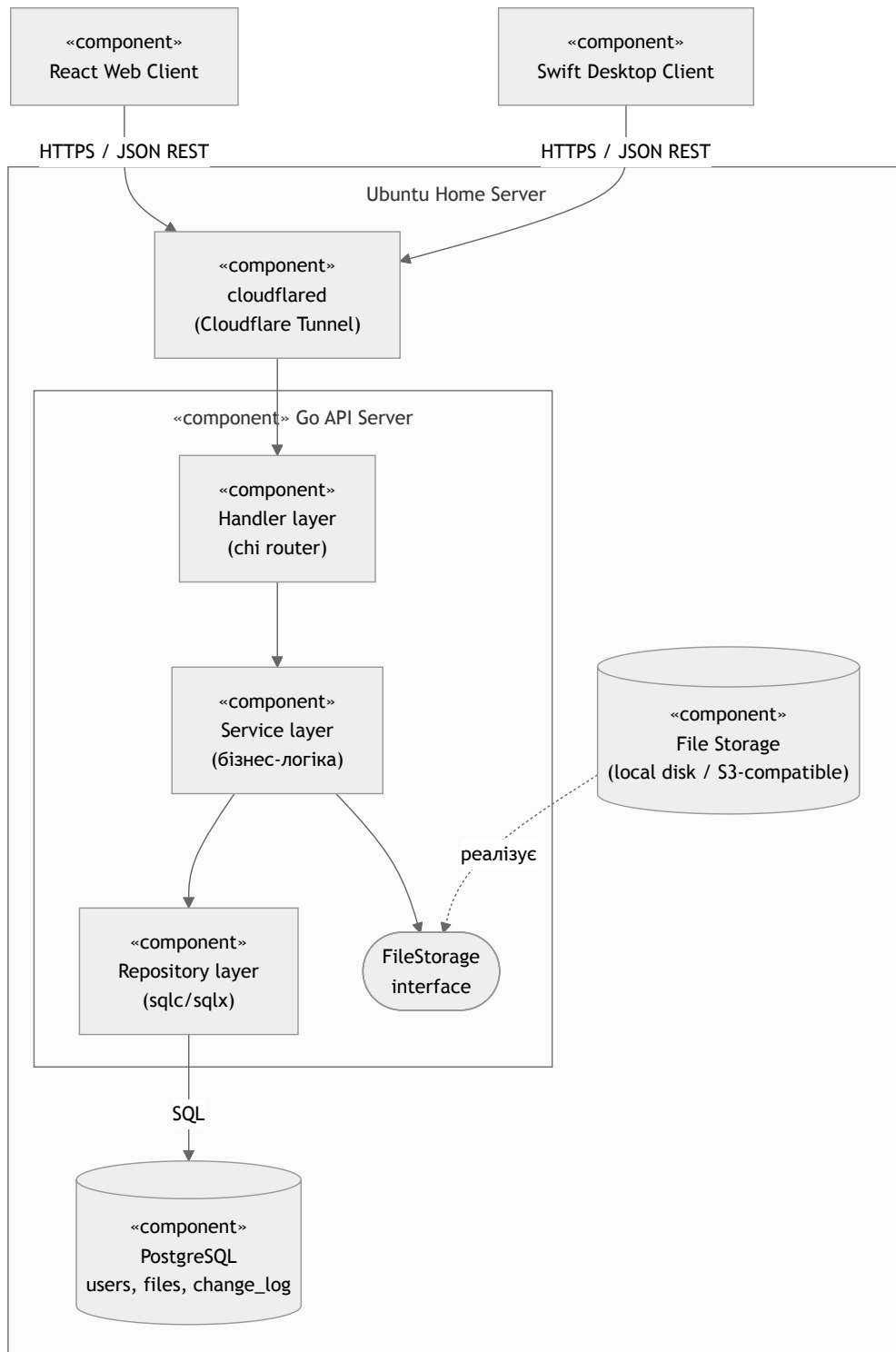


Рис. 11: Діаграма компонентів системи

3.6 Діаграма пакетів

Якщо діаграма компонентів (рис. 11) показує компоненти часу виконання та інтерфейси між ними, то діаграма пакетів (рис. 12) відображає *статичну* організацію коду за модулями відповідно до запланованої структури репозиторію (backend, frontend, desktop, migrations): у середині Go-модуля – шарова структура пакетів handler → service → repo/storage, що відповідає конвенції “жодних прямих викликів файлової системи з handler” та принципу доступу до сховища лише через інтерфейс FileStorage. Пакет геро додатково залежить від схеми, визначеної в migrations.

Примітка щодо нотації: Mermaid не має вбудованого типу діаграми пакетів, тому пакети зображено вкладеними subgraph-блоками (стереотип «package» у заголовку), а залежності – пунктирними стрілками з підписом дії.

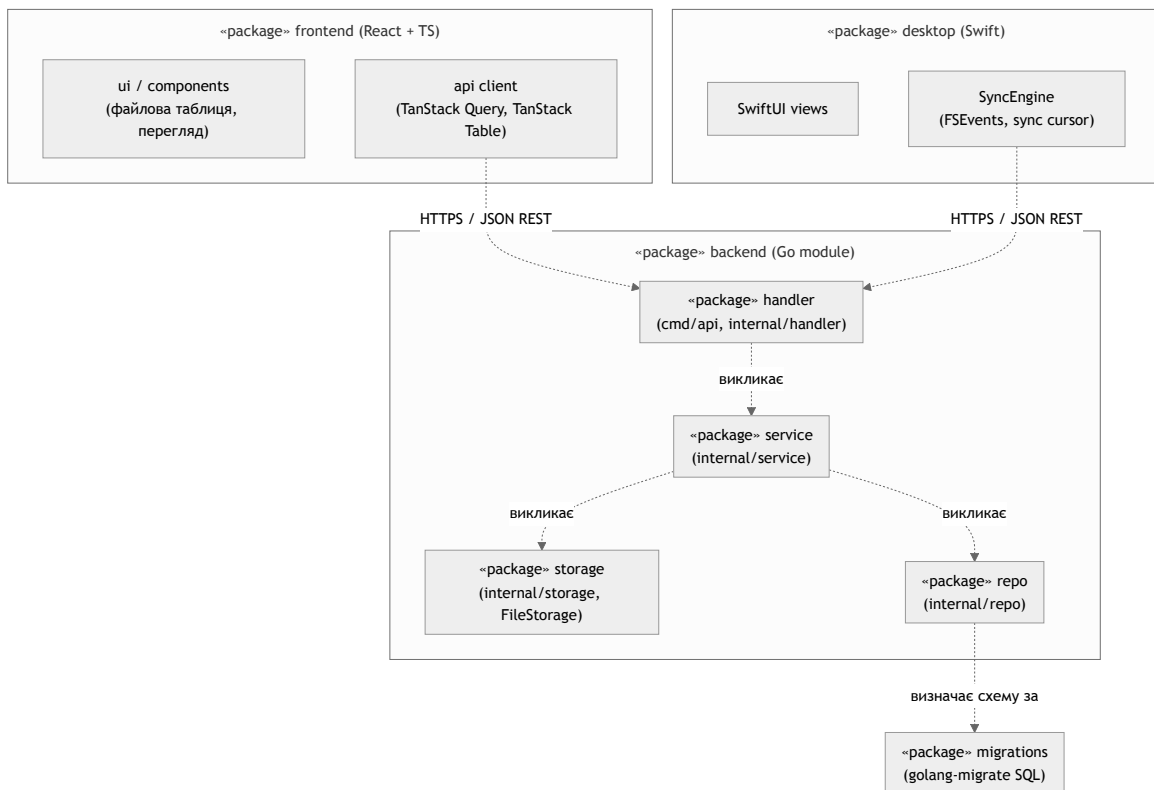


Рис. 12: Діаграма пакетів: організація коду за модулями

3.7 Діаграма активності

Рис. 13 деталізує бізнес-логіку одного циклу синхронізації: обчислення локального маніфесту, надсилання його на сервер, визначення дії для кожного файлу (upload/download/no-op/conflict/delete_local), виконання дії або, у разі конфлікту, отримання рішення від користувача (keep local / keep remote / keep both) і, зрештою, збереження нової версії курсора синхронізації (розділ 6 вимог).

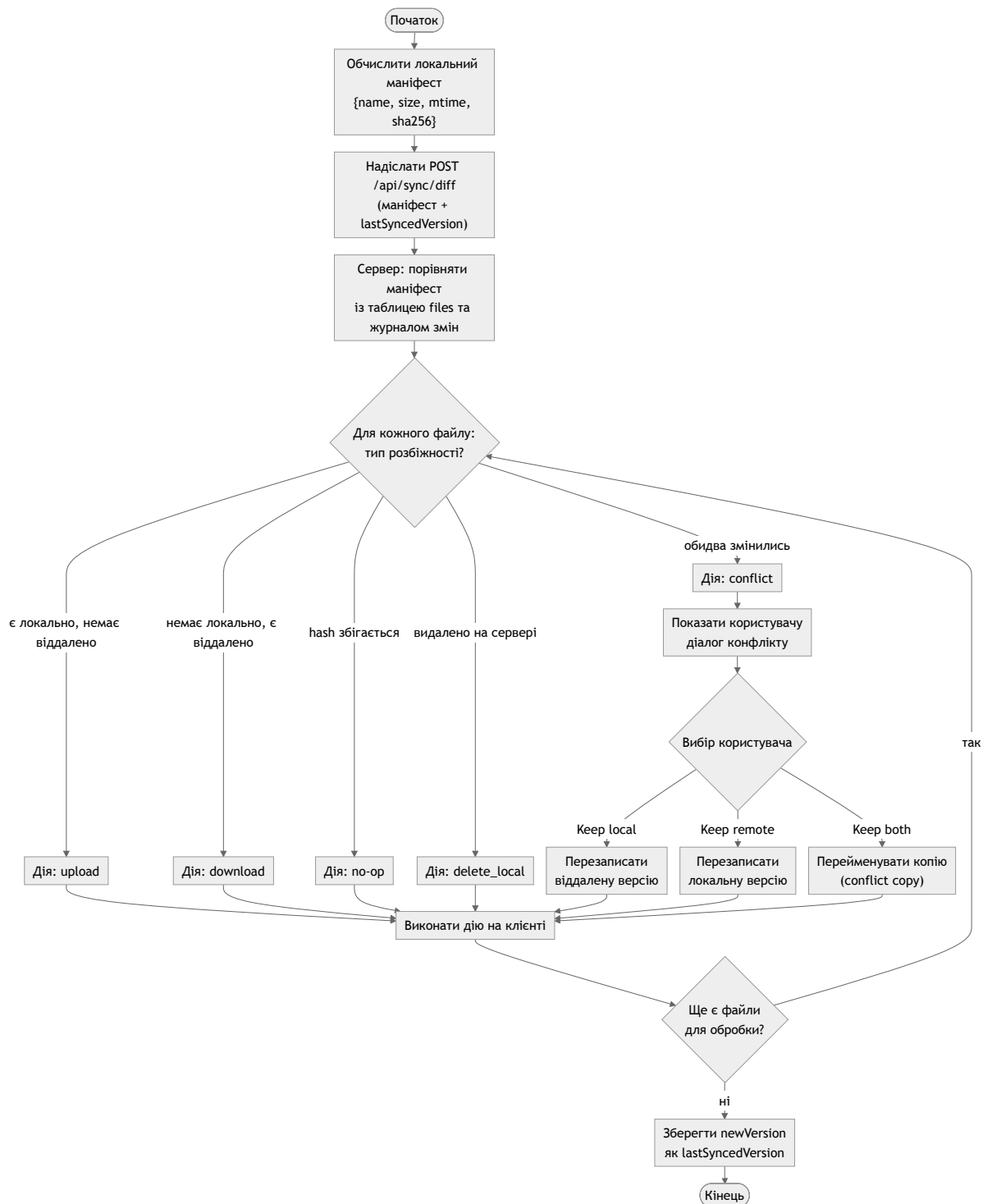


Рис. 13: Діаграма активності: цикл синхронізації

4. Висновки

Розроблений комплект діаграм послідовно розкриває систему Remote File Folder Client від бізнес-вимог до деталізованої структури і поведінки: діаграма прецедентів фіксує функціональні сценарії користувача; загальна діаграма класів і діаграма розгортання задають архітектурний каркас;

VOPC-діаграма, діаграма об'єктів, діаграми послідовності та комунікаційна, станів, компонентів, пакетів та активності деталізують реалізацію найскладнішого сценарію – синхронізації файлів між локальною та віддаленою текою, включно з протоколом порівняння маніфестів та розв'язанням конфліктів редагування. Зокрема, діаграма об'єктів і комунікаційна діаграма доповнюють відповідно VOPC-діаграму та діаграми послідовності конкретним знімком станів та альтернативною (структурною, а не часовою) нотацією того самого сценарію, а діаграма пакетів фіксує статичну модульну організацію коду, узгоджену з плановою структурою репозиторію. Побудовані діаграми узгоджені між собою (спільні класи, сутності та дії) та слугують основою для подальшої розробки й тестування системи.